Multi-class Image Classification Using a Convolutional Neural Network

Daniel Shklyarman

December 5, 2023

Contents

1 Introduction **2**

2 Machine Learning Background **2**
2.1 Neural Networks . . . 2
2.1.1 Loss Functions . . . 2
2.1.2 Activation Functions . . . 2
2.1.3 Optimization Functions . . . 3
2.2 Convolutional Neural Networks . . . 3
2.2.1 Convolutional Layers . . . 4
2.2.2 Pooling Layers . . . 4
2.2.3 Dense Layers . . . 4
2.2.4 Dropout Layers . . . 4

3 Methods **4**
3.1 Datasets . . . 4
3.2 Keras . . . 4
3.3 Layers . . . 5
3.4 Optimizer and Loss . . . 5
3.5 Epochs, Batchsize, and Train/Validation split . . . 6

4 Results **6**
4.1 Examples of Success and Failure . . . 7
4.1.1 Cat-Dog-Neither . . . 7
4.1.2 Cat-Dog-Fish . . . 8
4.1.3 Cat-Bird-Fish . . . 9
4.1.4 Cat-Dog-Bird-Fish . . . 10

5 Future Extensions **11**
5.1 Model Adjustment . . . 11
5.2 Expansion . . . 11

6 Bibliography **12**

1 Introduction

In order to gain familiarity with the theory of machine learning, and the tools used to apply it, I built multiple models that take a photo and classify it as one of several classes. The model is based on the Keras API and implements a Convolutional Neural Network in order to tell apart cat and dog images. It was built and run in Jupyter for VSCode and the fully trained models is about 75 percent accurate, depending on the class.

2 Machine Learning Background

2.1 Neural Networks

A neural network is a computational model inspired by the structure and function of the human brain. They are designed to recognize and model complex patterns and relationships in data.

A neural network consists of artificial neurons arrayed into layers. The layers are broadly categorized into the input layer, where the data is initially fed into the network, the hidden layers that process the input data, and the output layer which contains some usable output - typically a prediction or classification. Each neuron takes a set of input values, applies a linear function to them (multiplying by a given weight and adding a bias), and then applies a non-linear activation function to the result before passing it forward to the next neuron. This process of passing data through the neural network is known as forward propagation.

$$Z = \vec{W} \cdot \vec{X} + B,$$

$$A = activation(Z).$$

Where W represents the weight matrix, X represents the input matrix, and B represents the bias. Activation represents the activation function and A is what's passed on to the next layer.

During back propagation, the gradient of a loss function is taken with respect to each weight and bias, and the weights and biases are then updated based on these gradients through an optimization function in order to reduce the loss.

During training, the weights and biases are updated over and over again, until eventually the model begins to produce accurate results.

2.1.1 Loss Functions

A loss function is used to quantify the error or difference between the models output and the target values in supervised learning, which provides a measure of how well the model is performing. Thus, the smaller the loss, the better the model is performing.

There are various loss functions depending on the nature of the task. Classifying images into one of several categories is an example of image classification.. Therefore, the loss function used here is categorical cross-entropy.

2.1.2 Activation Functions

Activation functions introduce non-linearity into the network, allowing it model non-linear relationships in the data.

There are various activation functions that can be used, however, this project only uses the ReLu and Softmax activation functions. ReLu or rectified linear unit simply keeps inputs that are above 0 and sets any negative inputs to 0. The softmax activation function transforms a vector of real numbers into a probability distribution by exponentiating each element and normalizing the results, ensuring that the output values range between 0 and 1 and sum to 1.

$$ReLu : f(x) = max(0, x),$$

$$Softmax : f(x) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

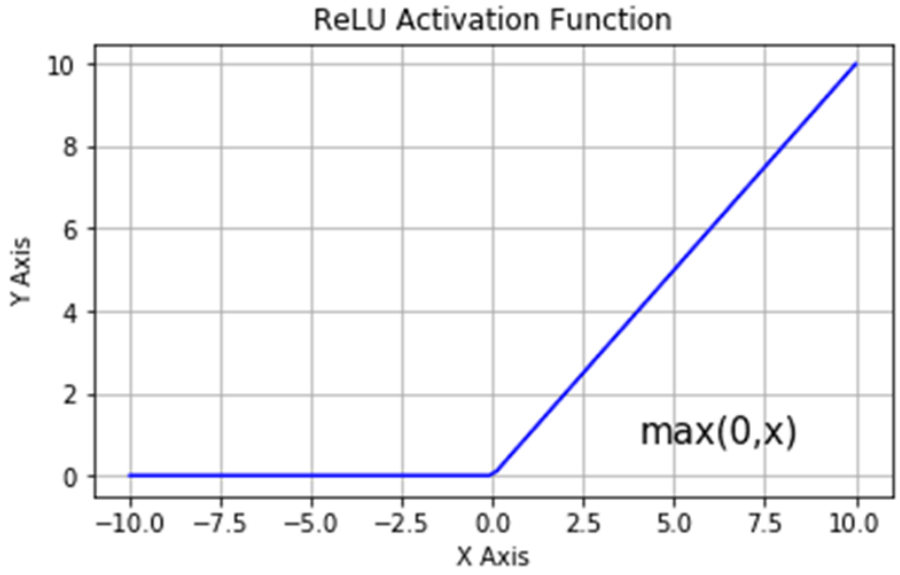


Figure 1: ReLU Graph

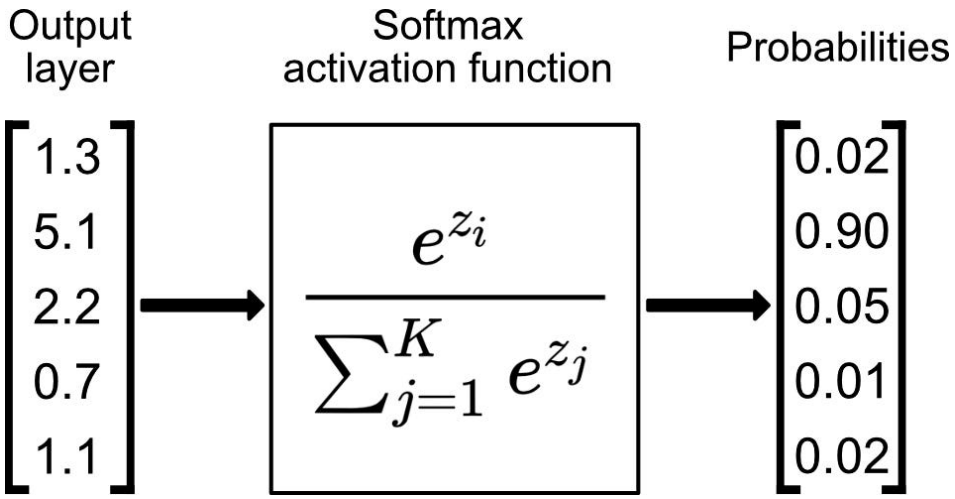


Figure 2: Softmax Visual

2.1.3 Optimization Functions

Optimization functions are used to iteratively update the weights and biases of each neuron in order to minimize loss. There are many different functions with each having different uses and benefits. The most basic is gradient descent, where at each iteration, the weights and biases are updated by subtracting the gradient of the loss function with respect to that weight/bias multiplied by a learning rate set at the creation of the model.

$$W = W - dw * \alpha,$$

$$B = B - db * \alpha.$$

2.2 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a specialized type of neural network used for processing structured grid data such as images. They include convolutional and pooling layers in addition to the fully connected layers used in standard neural networks.

2.2.1 Convolutional Layers

Convolutional layers consist of a number of filters that slide over the input data to detect patterns and features like edges and textures. A filter is matrix, typically 3x3xthe depth of the input, though other heights and widths are possible.

The convolutional operation is an element wise multiplication of the filter with the local region of the data, followed by a summation, which forms a map that highlights the presence of certain features within that region of the input.

During the training the process, the filters are trained to better detect the features in the data.

2.2.2 Pooling Layers

Pooling layers involve a tile (typically 2x2) being 'slid' over the input data and keeping either the maximum value of the local data or an average value from the local data. This serves to decrease the spatial dimensions of the data while keeping the essential information. The lower spatial dimensions reduce noise, prevent over-fitting, and ease the computational burden.

2.2.3 Dense Layers

A Dense layer, also known as a fully connected layer, is a layer within the network in which every node (or neuron) in the layer is connected to every node in the previous layer and every node in the following layer, information to flow between all nodes in adjacent layers. These are typically used to capture complex patterns in the training data, however their high resolution can make them prone to over-fitting.

2.2.4 Dropout Layers

Dropout layers are a regularization technique to help prevent over-fitting by randomly dropping some amount of the neurons (setting their output to 0) during training - 20 percent in this case. This prevents the model from becoming overly reliant on the output of specific neurons.

3 Methods

3.1 Datasets

For training the models, I used 5279 cat photos, 4976 dog photos, 4999 bird photos, 4948 fish photos, and 5279 'other' photos. The cat photos were collected from the Cats dataset. The dog photos were collected from the Stanford Dogs dataset. The bird photos were collected from the BIRDS 525 SPECIES- IMAGE CLASSIFICATION dataset. The fish images were taken from the Fish Dataset. The 'other' images were collected from kangaroo-dataset, Animal Image Dataset (90 Different Animals), cat vs rabbit, Lions or Cheetahs - Image Classification datasets.

For testing, 100 images of each class were collected. The cat images from cat vs rabbit, the dog images from Dog Emotions Prediction, the bird images from BIRDS 525 SPECIES- IMAGE CLASSIFICATION, the fish images from Fish Dataset, and the 'other' images from Animal Image Dataset (90 Different Animals), kangaroo-dataset, and Dolphin V Whale

The images were converted to 128x128 px for both training and testing and were all in full color, so the matrix representing each image was 128x128x3.

3.2 Keras

Keras is a Python based deep learning API used to streamline the process of building and training the model.

From Keras I used the ImageDataGenerator for both augmenting the images and for feeding them to the model during training. I also used the Sequential class, which groups a linear stack of layers into a model, and the Conv2D, MaxPool2D, Flatten, Dropout, and Dense layer classes.

3.3 Layers

The model is made up of 10 layers in sequence, with each layer having 1 input tensor and one output tensor.

The first 6 layers are a sequence of a convolutional layer followed by a pooling layer repeated 3 times. Each convolutional layer is made up of 3x3 filters, with the first layer having 16 filters, the second 32, and the third 64. The convolutional layers all use a ReLu activation function for the sake of simplicity as ReLu is the most generally useful activation function. Each pooling layer has a 2x2 tile and keeps only the greatest value in each tile, in order to reduce noise.

The seventh layer is a simple flattening layer that unrolls the tensor from the previous layer of shape (14,14,64) into a 1 dimensional tensor of shape (12544,). The output tensor can then be used in the following layers.

The eighth and tenth layers are both dense layers. The eighth layer has 512 nodes and uses ReLu as its activation, just like the convolutional layers. The ninth layer is a dropout layer that randomly sets the outputs of 20 percent of the nodes to 0. The tenth and final layer has as many nodes as there are classes and uses a softmax activation function in order to produce a a set of probabilities with each probability corresponding to the chance that the image is of a specific class.

Layer	Shape	Activation
Input	(128,128,3)	NA
Conv2D	(126,126,16)	ReLu
MaxPool	(63,63,16)	NA
Conv2D	(61,61,32)	ReLu
MaxPool	(30,30,32)	NA
Conv2D	(28,28,64)	ReLu
MaxPool	(14,14,64)	NA
Flatten	(12544,)	NA
Dense	(512,)	ReLu
Dropout	(512,)	NA
Dense	(num classes,)	Softmax

Table 1: Layer Summary.

3.4 Optimizer and Loss

The model uses Adaptive Moment Estimation (Adam) as the optimization function as it is the most generally used optimization algorithms, and easily implementable through Keras.

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{S_{dwt} - \epsilon}} * V_{dwt},$$

$$b_t = b_{t-1} - \frac{\eta}{\sqrt{S_{dbt} - \epsilon}} * V_{dbt}.$$

The model uses categorical cross-entropy to compute the loss. It measures the dissimilarity between predicted probability distributions and actual distributions, assigning higher penalties for larger deviations, guiding the model to minimize the differences between the predicted and true class probabilities.

$$Loss = -\sum_{i=1}^{NumClasses} y_i * log(\hat{y}),$$

$$y_i = TrueProbability,$$

$$\hat{y} = PredictedProbability.$$

3.5 Epochs, Batchsize, and Train/Validation split

The Cat-Dog-Neither model was trained with 20 epochs while the rest were trained with 15 due to time constraints.

The batch size was arbitrarily set to 512.

The training data was split into 80 percent for the actual training and 20 percent for validation.

4 Results

The models all achieve an accuracy of about 75-85 percent with all classes except dogs which consistently have a near random accuracy rate (about 1 in 3 accuracy for 3 class models and about 1 in 4 accuracy for the 4 class model). This is presumed to be because of the larger variety of dog images (i.e. multiple dogs, with owners) and larger differences between dog breeds as opposed to the more focused photos of other animals.

All the models appear to generally be most accurate with cats and fish. This is likely due to most cat breeds and fish species looking fairly similar.

Accuracy for each class was measured by taking a set of 100 testing images for each class and running those images through the model. The accuracy was then the number of images that the model predicted correctly divided by 100.

Model	Category	Accuracy	Average Accuracy
Cat-Dog-Neither	Cats	0.80	0.62
	Dogs	0.44	
	Neither	0.61	
Cat-Dog-Fish	Cats	0.7	0.64
	Dogs	0.32	
	Fish	0.89	
Cat-Bird-Fish	Cats	0.85	0.81
	Birds	0.74	
	Fish	0.85	
Cat-Dog-Bird-Fish	Cats	0.85	0.66
	Dogs	0.27	
	Birds	0.76	
	Fish	0.77	

Table 2: Model Accuracy.

4.1 Examples of Success and Failure

4.1.1 Cat-Dog-Neither



Figure 3: Success

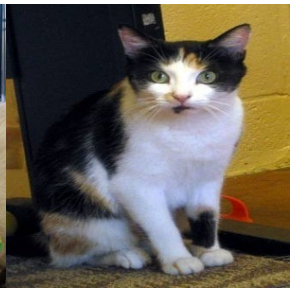


Figure 4: Success

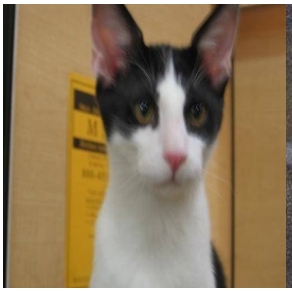


Figure 5: Failure



Figure 6: Failure



Figure 7: Success



Figure 8: Success



Figure 9: Failure



Figure 10: Failure



Figure 11: Success



Figure 12: Success



Figure 13: Failure



Figure 14: Failure

4.1.2 Cat-Dog-Fish



Figure 15: Success

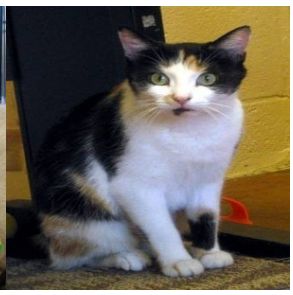


Figure 16: Success



Figure 17: Failure



Figure 18: Failure



Figure 19: Success



Figure 20: Success



Figure 21: Failure



Figure 22: Failure



Figure 23: Success



Figure 24: Success



Figure 25: Failure



Figure 26: Failure

4.1.3 Cat-Bird-Fish

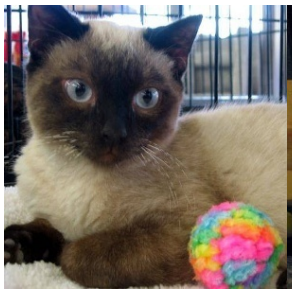


Figure 27: Success

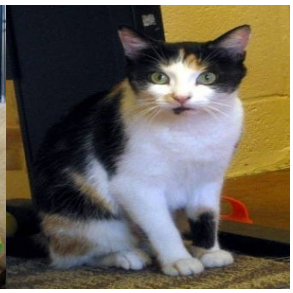


Figure 28: Success



Figure 29: Failure



Figure 30: Failure



Figure 31: Success



Figure 32: Success



Figure 33: Failure



Figure 34: Failure



Figure 35: Success



Figure 36: Success



Figure 37: Failure



Figure 38: Failure

4.1.4 Cat-Dog-Bird-Fish



Figure 39: Success

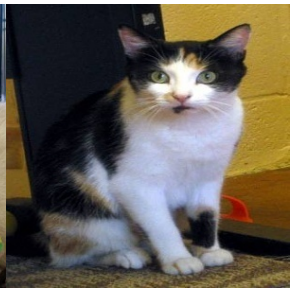


Figure 40: Success



Figure 41: Failure



Figure 42: Failure



Figure 43: Success



Figure 44: Success



Figure 45: Failure



Figure 46: Failure



Figure 47: Success



Figure 48: Success



Figure 49: Failure



Figure 50: Failure



Figure 51: Success



Figure 52: Success



Figure 53: Failure



Figure 54: Failure

5 Future Extensions

5.1 Model Adjustment

Many of the models hyper-parameters were set arbitrarily and not experimented with due to time constraints. Future work could involve fine-tuning the model by adjusting layer size, kernel size, number of layers, learning rates, batch size, number of epochs, and trying new types of layers and activation functions.

Additionally, the model is theoretically capable of handling any number of classes and future work would involve testing the model on 5, 6, 7, etc classes. These additional classes could also be things besides animals, like landscapes or appliances. There could also be some work done on figuring out how 'Not a known class' could be implemented as a potential output for the model.

Finally, the model should also be tested when trained on different datasets, especially a different dog dataset, as the current model or dog dataset results in extremely poor accuracy.

5.2 Expansion

At the moment the model is relatively simple, only outputting a prediction of a given image's class. Future work could involve locating animals within an image, or, more generally, identifying which part of an image is responsible for the model's prediction. Additionally, the project could be expanded to identify multiple classes within a single image, like an image of a cat and a dog, or an image of a happy dog and another of a sad cat.

6 Bibliography

References

[IBMa] What are convolutional neural networks?

[IBMb] What are neural networks?

[LBH15] Yann LeCun, Y. Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–44, 05 2015.

[Mus22] Musstafa. Optimizers in deep learning, Feb 2022.

[Nav23] Naveen. What is relu and sigmoid activation function?, Aug 2023.

[Tal23] KGP Talkie. Deep learning tutorial 4 - cats vs dogs classification using cnn and image augmentation, Dec 2023.

[WKG+19] Jorg Wagner, Jan Mathias Kohler, Tobias Gindele, Leon Hetzel, Jakob Thaddaus Wiedemer, and Sven Behnke. Interpretable and fine-grained visual explanations for convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.